Fractional Flow Reserve derived from coronary CT angiography

What is Fractional Flow Reserve (FFR)?

Fractional Flow Reserve is an **invasive medical test** that measures how much a narrowed coronary artery limits blood flow to the heart. It requires inserting a **catheter with a pressure sensor** into the arteries during angiography to determine whether a patient needs treatment.

Angiography

Angiography involves injecting a **special dye (contrast agent)** into the bloodstream and then taking **X-ray images** to see how blood flows through the vessels. It's often used to check for **blockages, narrowings (stenosis), or abnormalities** in the heart’s arteries.

1. A **catheter** is inserted (usually into your wrist or groin).
2. The dye is released through the catheter.
3. X-ray images are taken to watch the flow of the dye through your heart's blood vessels.

What is Fractional Flow Reserve Computed Tomography (FFRCT)?

FFRCT is a **non-invasive method** that estimates FFR values using **coronary CT scans combined with advanced computer models**, including artificial intelligence or computational fluid dynamics. It helps doctors assess whether blockages in the arteries are restricting blood flow — without the need for inserting any instruments into the body.

What is Computational Fluid Dynamics (CFD)?

CFD is a branch of engineering and physics that uses **mathematical models and computer simulations** to study how **fluids (like blood, air, or water) flow**.

Overview

While invasive FFR relies on pressure wire measurements obtained during catheterization, traditional FFRCT estimates these values non-invasively by simulating coronary blood flow using computational fluid dynamics (CFD). However, the high computational demands of CFD have motivated the development of AI-based FFRCT methods, which aim to streamline the process while maintaining diagnostic accuracy.

Understanding the problems of FFR

01 It’s invasive: requires inserting a catheter and using pressure wires.

02 It involves risks like bleeding, infection, or vessel damage.

03 It can be expensive and time-consuming.

Solving problems of FFR by using FFRCT

01 Non-invasive: uses a CT scan and AI/computational models.

02 Safer, no catheterization needed.

03 Faster, especially with AI accelerating estimation.

CT Scan

A **CT scan** (**Computed Tomography scan**) is a medical imaging technique that combines **X-rays** and computer processing to create **detailed cross-sectional images** of the inside of your body.

What it does:

- Shows **bones, organs, blood vessels, and soft tissues** more clearly than regular X-rays.
- Allows doctors to **see slices of your body**

Digital twin-based simulation frameworks

A digital twin-based simulation framework is a virtual copy of something in the real world that you can test, monitor, and improve without touching the real thing.

How it works:

Make a digital copy → using data, measurements, and models of the real thing (e.g., a heart, a factory, an airplane).

Feed it real-world data → sensors, scans, or historical records keep it updated so it matches the real object.

Run simulations → test “what if” scenarios without touching the real thing (What if we put a stent in this artery? What if the patient’s blood pressure is lowered?)

Medical example – FFRCT:

Digital twin of a patient’s heart and coronary arteries made from CT scans

Use physics-based and AI models to simulate blood flow

Doctors can see how changes might affect the patient before doing the procedure

Why it’s useful:

Safer: No risk to the patient during testing

Faster: No need for invasive procedures to try different scenarios

More personalized: It’s based on your exact anatomy and data

The Problem with Traditional CFD in FFRCT

While CFD gives accurate FFRCT results, it has major drawbacks:

1. **Slow:** It can take hours to days to simulate blood flow.
2. **Heavy Computation:** Needs supercomputers or powerful servers.
3. **Complex:** Requires detailed modeling of heart vessels and blood physics.

How AI Accelerates FFRCT

While CFD gives accurate FFRCT results, it has major drawbacks:

1. **Quick:** It Predicts FFRCT values from CT scans quickly, this reduces processing time from hours to seconds/minutes.

2. **Inexpensive:** No need for expensive hardware or high-level CFD expertise.

3. **Efficiency:** Faster results = more patients analyzed in less time.

CFD-based FFRCT vs AI-based FFRCT

Aspect	CFD-based FFRCT	AI-based FFRCT
Speed	Hours to days	Seconds to minutes
Hardware	Supercomputer	Regular Computer
Accuracy	High	Near high
Scalability	Low	High
Cost	Expensive	More Affordable

Hybrid methods combining AI and reduced-order CFD for FFRCT

Why combine them?

Full CFD is very accurate but slow and computationally expensive (can take hours per patient). AI alone is very fast but may lack some physics knowledge and generalizability. Reduced-order CFD simplifies physics equations to make simulation faster but still physically meaningful. Hybrid methods merge AI's speed with reduced-order CFD's physical accuracy.

How does it work?

Use reduced-order CFD to model blood flow quickly but approximately. Train an AI model on many reduced-order CFD simulations plus real patient data. The AI learns to predict FFRCT from patient CT scans using patterns informed by CFD physics. At inference, the AI can give near-CFD-accurate results very fast, suitable for clinical use.

Common AI Approaches Used

- **CNNs**: Neural model that analyzes image data by applying filters to automatically detect visual features like edges, textures, or shapes.

- **GNNs**: Neural model designed to analyze data represented as graphs by learning how connected nodes influence each other through their relationships.

- **MLPs**: Neural network that processes structured input data through fully connected layers to learn patterns and make predictions.

Convolutional Neural Networks (CNN)

- **What it does**: CNNs are really good at analyzing images
- **How it helps in FFRCT**: It looks at the structure of arteries and automatically detects patterns (like narrowing or plaque) that could affect blood flow, based on what it learned from many previous scans.
- **Example Use**: Image classification, medical imaging, facial recognition.

1. **Input Layer:** This is where the CNN receives an image (image is usually a grid of numbers)

2. **Convolutional Layer:** Small "filters" (like 3x3 windows) slide over the image. It detects features like edges, shapes, and textures. At each step there is an output a number (like a score for how much the pattern matched the filter). These numbers form a new image, called a feature map.

Feature Map: 2D matrix representing the output of a convolutional layer. It highlights specific features learned from the input image by a particular filter, each feature map corresponds to a specific filter.

3. **Activation Layer:** After filtering, the CNN uses an activation function (like ReLU) to keep only the important signals (values that show a filter detected something useful). ReLU helps by zeroing out unhelpful values, making learning faster and more accurate. - (filter weak signals)

4. **Pooling Layer:** The CNN then shrinks the feature maps using max-pooling or average-pooling — to keep only the strongest features, so data is smaller and faster to process. - (Reduces size of image - downsampling)

5. **More Convolution + Pooling Layers:** This is done multiple times — the network keeps learning deeper patterns. Early layers might detect simple edges, later layers detect full shapes.

6. **Flattening Layer:** taking the multi-dimensional data (like 2D or 3D feature maps from convolution and pooling layers) and converting it into a 1D list.

7. **Fully Connected Layers:** These are standard neural network layers, they process the flattened features and learn what they mean.

8. **Output Layer:** Produces the final prediction (e.g., Yes/No for disease)

Input Type

CNNs work best with data that has **spatial structure** — meaning the data has dimensions where **nearby values are related.**

Input Type:

- Images
- Videos
- Spectrograms
- Genomic sequences

Output Type Example

Single value (per point):

`FFRCT = 0.73` at a specific location in the coronary artery — suggests **possible ischemia** (FFR < 0.80 is usually abnormal)

Vector (along artery):

`[0.92, 0.87, 0.81, 0.76, 0.68]` — FFRCT values **along the vessel**, showing a **drop in pressure** downstream

Convolutional Neural Networks (CNN)

Pro’s	Con’s
Great at analyzing image data	Needs a lot of labeled image data to train
Highly accurate in image recognition tasks	Performance drops if image quality is poor
	Doesn't understand relationships between structures

Graph Neural Networks (GNN)

- **What it does**: GNNs analyze data structured like a graph—where objects (like blood vessels) are **nodes**, and their connections (like how vessels branch) are **edges**.

- **How it helps in FFRCT**: Because blood vessels form a network, not a flat image. GNNs can model the relationship between different parts of a vessel tree. That's something CNNs struggle with. For example, if one segment is blocked, GNN can reason about how it affects other parts of the network.

- **Example**: A GNN sees that vessel A connects to B and C, with a narrowing between A and B, and uses this to estimate flow at each point.

1. **Input graph structure:** Each node can have features, and each edge can have connections.

2. **Message Passing:** Each node gathers info ("messages") from its neighbors. A node checks what features its neighbors have, and It updates itself based on that info.

3. **Multiple Layers = Deep Learning:** This process is repeated over many layers (iterations). So, a node will learn not just about its direct neighbors — but also the neighbors of neighbors (global structure).

4. **Final Output:** After enough message passing, you can predict something for each node (e.g., is this point blocked?) and predict something for the entire graph (e.g., simulate flow through the vessel network).

Input Type

The **best input for a Graph Neural Network (GNN)** is **graph-structured data**, where both the relationships (edges) and the properties (features) of nodes or edges are important.

Ex:

Domain	Node	Edge	Node Features
Social Networks	Users	Friendships	Age, interests
Molecule Modeling	Atoms	Chemical bonds	Atom type
Recommendation	Products or users	Co-purchases or reviews	Ratings

Output Type

1. Node Classification

What it is: Predicting a label for each node (point) in a graph.

Example: In a social network graph, predict the interest group or age category of each user (node).

Output: A label (e.g., "sports lover") for every node.

2. Graph Classification

What it is: Predicting a single label for the entire graph.

Example: Predict if a molecule (represented as a graph of atoms and bonds) is toxic or non-toxic.

Output: One class label per graph (e.g., “toxic”).

3. Link Prediction

What it is: Predicting whether a link between two nodes should exist.

Example: In a recommendation system, predict if user A will connect with user B or buy product X.

Output: A probability or score indicating the likelihood of a new connection.

Graph Neural Networks (GNN)

Pro’s	Con’s
Preserve structure and relationships	Lack of standardized architectures
Better at representing real anatomy as graphs	Needs accurate graph creation
Useful when data is structured	Computationally expensive

Multi-Layer Perceptrons (MLP)

- **What it does**: MLPs are standard feedforward neural networks that take in numerical features and give a prediction.

- **How it helps in FFRCT:** After features are extracted (manually or using CNN/GNN), the MLP combines them to estimate the FFR value and classify patients risk.

- **Example:** An MLP might take in features like vessel radius, length, and wall thickness and predict whether blood flow is likely reduced.

Input Type

MLPs input type needs to be flattened into vectors first, they do not take in structured spatial data like images (as 2D grids) or graphs natively.

Ex:

- A row from a spreadsheet (like age, heart rate, cholesterol level, etc.)
- Flattened image data (e.g., turning a 28×28 image into a 784-length vector)
- Encoded categorical values (like gender = 0/1, country = 3 - the process of converting categorical variables into numeric features)

Output Type

1. Node Classification

What it is: Predicting a label for each node (point) in a graph.

Example: In a social network graph, predict the interest group or age category of each user (node).

Output: A label (e.g., "sports lover") for every node.

2. Graph Classification

What it is: Predicting a single label for the entire graph.

Example: Predict if a molecule (represented as a graph of atoms and bonds) is toxic or non-toxic.

Output: One class label per graph (e.g., “toxic”).

3. Link Prediction

What it is: Predicting whether a link between two nodes should exist.

Example: In a recommendation system, predict if user A will connect with user B or buy product X.

Output: A probability or score indicating the likelihood of a new connection.

1. Input Preparation: Flatten your input data into a 1D vector (e.g., [age, cholesterol, flow rate, artery diameter] and normalize values (value between 0 and 1) because if one input (like “cholesterol = 240”) is much bigger than another (like “age = 20”), the model might wrongly think cholesterol is more important just because it has a bigger number.

2. Define the MLP Architecture:

- **Input layer size =** number of features
- **Hidden layers =** 1 or more layers with neurons (e.g., 64, 128)
  More neurons = the model can learn more complex patterns.
  But too many neurons = more computation and risk of overfitting.

- **Activation function:** It introduces non-linearity into the network — meaning the model can learn complex patterns, not just straight lines. Usually ReLU for hidden layers, and softmax (for multi-class classification) or sigmoid (for binary classification) for output.

Binary Classification: You're choosing between two classes only.

Output: Usually just 1 neuron in the output layer.
Use sigmoid activation, which gives a probability between 0 and 1.

Multi-Class Classification: you're choosing between 3 or more classes.

Output:
You have one output neuron per class.
Use softmax activation, which turns scores into probabilities that add up to 1.

Without non-linear activations, your multi-layer perceptron (MLP) is basically just one big linear equation — no matter how deep you make it. Activation functions (like ReLU, sigmoid) break that limitation and allow MLPs to model real-world patterns that aren't straight lines.

ReLU - if input > 0, keep it. If not, output 0
Sigmoid - squashes number to a probability, value between 0 and 1
Softmax - Turns scores into probabilities that add up to 1

In MLP:

Each layer does linear math (weights * input + bias) then Non-linear activation (like ReLU)

This combo allows the MLP to learn complex relationships in your data.
Without non-linearity, it would be useless for real problems.

- Output layer size = depends on task (1 for regression and activation is usually linear or none, 2 for predicting between 2 classes, and for multiple classification, output layer size is equal to number of classes)

3. Training the MLP

1. Use a labeled dataset (inputs + correct output) to help it learn from its mistakes

Set:

2. Loss function: this tells the MLP how wrong it was.

   - MSE (Mean Squared Error) → for numbers (like predicting FFR)
   If the real FFR is 0.8 and the model says 0.6, the loss is high.

   - Cross-Entropy → for choices (like yes/no, class A/B/C)
   Smaller loss = better predictions.

3. Optimizer: method to improve the model, it updates the weights of the MLP to decide how much to adjust to reduce the loss.

Common choices:

SGD (Stochastic Gradient Descent) – Takes small, simple steps to fix the model. Reliable, but slower.

Adam – A smart optimizer that adapts its steps. Faster, better for most problems.

4. Epochs: 1 full round through all your data.

If you have 1000 patients, one epoch = the model sees all 1000 once.
Usually, we train for 10–100 epochs to improve step by step.

5. Batch Size: you don’t show the model all the data at once. Instead, you give it small chunks.

Batch size = 32 → the model trains on 32 rows at a time.

Smaller batches = slower but more accurate learning.
Bigger batches = faster but may miss fine details.

5. Backpropagation

Calculates loss: measures how wrong the prediction is
Backpropagation: traces that error backward through the network
Gradient Descent: adjusts weights in the right direction, slowly

MLP compares output to real label → calculates loss
Adjusts weights using gradient descent to reduce the error

6. Evaluation: after training, test the model on unseen data

1. Give it new inputs
Example: new patient data, new CT scan numbers, new house details.

2. Let the MLP make predictions
It outputs its guess — e.g., "disease risk = 80%" or "price = $200,000".

3. Compare predictions with the real answers

4. Score it using numbers:

Classification problems (Yes/No, categories):

Accuracy → % of correct predictions
Precision → Out of the cases it said "Yes", how many were actually Yes

Regression problems (predicting numbers):

Mean Absolute Error → Average difference between guess and real number
Root Mean Squared Error → Like MAE but punishes big mistakes more

7. Prediction: Use the trained MLP to make new predictions

Multi-Layer Perceptrons (MLP)

Pro's	Con's
Simple and flexible: Can handle many types of problems (classification, regression)	Requires **flattened input** (not good with images, sequences unless preprocessed)
Learns complex patterns with enough layers and data	Needs a **large amount of data** to perform well
Works well with **tabular data** and **engineered features**	Can **overfit** easily without regularization or enough data

Strengths, Limitations, and Clinical Relevance of Each Method

1. Convolutional Neural Networks (CNNs)

Best for: Image-based tasks (e.g., CT scans)

Strengths:

- Excellent at extracting spatial features from coronary CT images
- Automatically detects vessel stenosis and plaque features
- Pretrained models can be fine-tuned for FFRCT

Limitations:

- Struggles with non-Euclidean (non-linear) structures like complex vessel trees
- Requires large amounts of labeled data
- Doesn’t naturally model inter-vessel relationships

Clinical Relevance:

- Useful in **lesion detection**, **stenosis assessment**, and **image segmentation**
- Common in current FFRCT pipeline to extract imaging biomarkers (measurable indicator of a biological state)

2. Graph Neural Networks (GNNs)

Best for: Modeling vessel trees

Strengths:

- Understands how the blood vessels are connected and how blood flows through them
- More physiologically relevant for simulating blood flow
- Can integrate image + anatomical graph data

Limitations:

- Harder to train; needs graph-structured input (e.g., segmented vessels)
- Less interpretability than CNNs or MLPs
- Still emerging in medical imaging research

Clinical Relevance:

- Promising for **personalized FFR prediction** based on entire artery structure
- Allows simulation of pressure drop across the coronary tree

3. Multi-Layer Perceptrons (MLPs)

Best for: Tabular data (e.g., clinical features, extracted image features)

Strengths:

- Simple, fast, and interpretable
- Effective when trained on hand-crafted features (e.g., plaque size, stenosis %)
- Combines well with outputs of CNNs or GNNs

Limitations:

- Cannot process spatial or structural data directly
- Depends heavily on feature engineering (process of creating, selecting, transforming, or scaling input data (features) so a machine learning model can understand it better and perform more accurately)
- Less accurate alone than CNN or GNN in complex tasks

Clinical Relevance:

- Useful for **risk scoring**, **FFRCT classification**, or combining imaging + clinical data
- Common in clinical decision support systems